

# Projects of operating systems

## 1. **Basic Shell Implementation**

### - **Objective**:

Implement a command-line interface (CLI) that allows users to execute built-in or external commands, handle background processes, and provide output redirection.

### - **Key Concepts**:

- Process creation using `fork()`, `exec()`, and `wait()`.
- Input/output redirection (`<`, `>`).
- Signal handling (`SIGINT`, `SIGCHLD` for background processes).
- Piping commands (`|`).

### - **Detailed Steps**:

1. Parse user input.
2. Implement built-in commands like `cd`, `exit`.
3. Use `fork()` and `exec()` to execute other programs.
4. Add support for background processes (using `&`).
5. Implement I/O redirection for commands.
6. Implement piping between commands (`command1 | command2`).

### - **Possible Extensions**:

- Job control (allowing users to bring background jobs to the foreground).
- Support for environment variables (`$PATH`).
- Command history with navigation.

---

## 2. **Thread Synchronization (Dining Philosophers Problem)**

### - **Objective**:

Solve the classic synchronization problem where multiple philosophers try to pick up shared resources (forks) to eat, leading to deadlocks and starvation issues.

- **\*\*Key Concepts\*\*:**

- Threads (using `pthread` or equivalent libraries).
- Mutexes and Semaphores for resource management.
- Deadlock detection and prevention.

- **\*\*Detailed Steps\*\*:**

1. Create a program where each philosopher runs as a separate thread.
2. Use mutexes or semaphores to manage access to shared resources (forks).
3. Implement locking and unlocking mechanisms to avoid deadlock.
4. Solve the problem of starvation.

- **\*\*Possible Extensions\*\*:**

- Implement deadlock prevention (e.g., resource hierarchy).
- Use monitors (a higher-level synchronization primitive).
- Add logging for visualizing the behavior of each philosopher.

---

### 3. **\*\*Virtual Memory Simulation\*\***

- **\*\*Objective\*\*:**

Create a simulator that demonstrates how a virtual memory system works, including paging and page replacement algorithms.

- **\*\*Key Concepts\*\*:**

- Paging, page tables, and page frames.
- Page replacement algorithms (FIFO, LRU, Optimal).

- **\*\*Detailed Steps\*\*:**

1. Implement a page table structure that maps virtual addresses to physical addresses.
2. Simulate memory access by loading pages into a limited set of frames.
3. Implement different page replacement algorithms.
4. Measure page fault rates for each algorithm.

- **\*\*Possible Extensions\*\*:**

- Implement a Translation Lookaside Buffer (TLB) for faster address translation.
- Add support for multi-level paging.
- Simulate different memory access patterns and compare performance.

---

## ### 4. **\*\*File System Design\*\***

### - **\*\*Objective\*\***:

Design and implement a simplified file system that supports essential operations like creating, deleting, reading, and writing files.

### - **\*\*Key Concepts\*\***:

- Inodes, file blocks, and directory structures.
- File allocation methods (contiguous, linked, indexed).

### - **\*\*Detailed Steps\*\***:

1. Design a data structure for file metadata (inodes).
2. Implement file creation and deletion.
3. Write a method to read and write data to files.
4. Manage file blocks on disk (use a file as a virtual disk).
5. Implement directories and subdirectories.

### - **\*\*Possible Extensions\*\***:

- Implement file permissions (read, write, execute).
- Add support for journaling (logging changes before applying them).
- Implement disk quotas for users.

---

## 5. **Process Scheduling Simulation**

### - **Objective**:

Simulate and compare different CPU scheduling algorithms to understand how they impact process management and system performance.

### - **Key Concepts**:

- CPU scheduling algorithms (Round-Robin, Priority, Shortest Job First).
- Context switching and CPU burst times.

### - **Detailed Steps**:

1. Create a queue to simulate the ready queue of processes.
2. Implement each scheduling algorithm.
3. Simulate process execution, including context switching.
4. Track performance metrics (e.g., average wait time, turnaround time).

### - **Possible Extensions**:

- Add context switching overhead and analyze its impact.
- Implement multi-level feedback queues (used in modern OS).
- Visualize scheduling with a Gantt chart.

---

## 6. **Memory Management with Dynamic Partitioning**

### - **Objective**:

Implement memory allocation strategies that simulate dynamic partitioning in a system's memory.

### - **Key Concepts**:

- Dynamic memory allocation strategies (First-Fit, Best-Fit, Worst-Fit).
- External fragmentation and memory compaction.

### - **Detailed Steps**:

1. Simulate a process that requests variable amounts of memory.
2. Implement First-Fit, Best-Fit, and Worst-Fit algorithms to allocate memory.

3. Track memory usage and fragmentation.

- **Possible Extensions:**

- Implement memory compaction to reduce fragmentation.
- Add a mechanism for memory deallocation and reallocation.
- Measure and visualize fragmentation over time.

---

## 7. **Disk Scheduling Algorithms Simulation**

- **Objective:**

Simulate disk scheduling algorithms to understand how they optimize seek time and improve I/O performance.

- **Key Concepts:**

- Disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN).
- Seek time and rotational latency.

- **Detailed Steps:**

1. Create a simulation where requests are placed for disk sectors.
2. Implement each disk scheduling algorithm.
3. Measure performance metrics (total seek time, average seek time).

- **Possible Extensions:**

- Simulate real-time priorities for certain I/O requests.
- Implement scheduling for SSDs (Solid-State Drives).
- Visualize head movement on a disk platter.

---

## 8. **\*\*Deadlock Detection and Prevention\*\***

### - **\*\*Objective\*\***:

Simulate a system that detects and prevents deadlocks in resource allocation, using real-world algorithms like Banker's Algorithm.

### - **\*\*Key Concepts\*\***:

- Resource Allocation Graph (RAG).
- Deadlock prevention (Banker's Algorithm).

### - **\*\*Detailed Steps\*\***:

1. Simulate processes requesting resources.
2. Implement a resource allocation graph to track resource ownership.
3. Implement deadlock detection algorithms.
4. Implement Banker's Algorithm for deadlock avoidance.

### - **\*\*Possible Extensions\*\***:

- Add resource preemption strategies.
- Implement deadlock recovery (e.g., terminating or rolling back processes).
- Simulate a variety of deadlock scenarios.

---

## 9. **\*\*User-Level Thread Library\*\***

### - **\*\*Objective\*\***:

Implement a thread library that provides basic threading functionalities like thread creation, synchronization, and joining.

### - **\*\*Key Concepts\*\***:

- Threading, context switching, and scheduling.
- Synchronization primitives (mutexes, semaphores).

### - **\*\*Detailed Steps\*\***:

1. Implement a thread control block (TCB) to manage thread states.
2. Use `setjmp()` / `longjmp()` to perform context switching between threads.

3. Implement mutexes for synchronization.
4. Implement basic thread scheduling (round-robin or priority-based).

- **Possible Extensions:**

- Implement a thread pool for managing a large number of threads.
- Add support for condition variables and barriers.
- Implement preemptive threading and context switching using timers.

---

## 10. **File Compression Utility**

- **Objective:**

Create a file compression and decompression utility using a well-known algorithm like Huffman Coding.

- **Key Concepts:**

- Data compression algorithms.
- File I/O and bit-level manipulation.

- **Detailed Steps:**

1. Implement Huffman Coding to compress files.
2. Store compressed files in a binary format.
3. Implement the decompression algorithm to recover the original file.

- **Possible Extensions:**

- Implement multiple compression algorithms (e.g., Run-Length Encoding).
- Add error detection mechanisms (like parity bits).
- Compare the compression ratio and time efficiency for different algorithms.